10-Minute English PFE Defense

Presenter memory sheet · target delivery 9:15 · 45-second safety margin

One-sentence thesis
I built a personalized, context-aware next-POI engine, decoded it without retraining into loop-free itineraries, and evaluated each task on its own fair benchmark.

0.187	0.289 > 0.259	0.021
HR@1 among ~5,100 NYC candidates	Frozen Rollout beats Trained Pointer	maximum Random gap vs Chen 2016

The rule to memorize

Compare results only when the dataset, protocol, and metric are identical.
Never compare next-POI HR@k with itinerary pairs-F1, and never compare NYC itinerary scores directly with Flickr literature scores.

Slide	Time	Memory cue
1	0:25	5,100 places to one route
2	0:45	What to visit + in what order
3	0:55	One engine, one decoder, two evaluations
4	1:15	Graph + context + user + GRU
5	0:50	HR@1 = 0.187; honest tier
6	1:15	Score, mask, select, repeat
7	1:05	0.289 > 0.259; supervision density
8	0:55	Same benchmark; max gap 0.021
9	1:10	Published scale + cold-start
10	0:40	Repeat the three anchors

If running late: shorten slides 8 and 9, but keep the comparability rule. Safe closing: *Under sparse trajectory data, decoding a densely supervised next-POI engine can outperform a dedicated itinerary model.*

Full 9:15 Talk Track

Pronunciation: say 'HR at one', 'HR at ten', and 'pairs F-one'. Each slide includes one sentence to memorize, a transition, and a safe sentence to skip.

Slide 1 · 0:25 · Smart Visit: From 5,100 Places to One Coherent Itinerary

MEMORY: Smart Visit turns 5,100 places into one personalized route.

Good morning. Smart Visit turns about 5,100 candidate places into one personalized, ordered itinerary. I will show the prediction engine, the simple route decoder, and the two separate evaluations that keep the conclusions fair.

Transition: *First, what does the traveler need?*

Skip if late: *It is a reproducible contribution, not a record claim.*

Slide 2 · 0:45 · A Good Trip Answers Two Questions

MEMORY: Next-POI asks what comes next; itinerary generation asks what order works.

A traveler needs more than attractive places. They need to know what to visit and in what order. First, which place is most likely to come next? Second, can repeated choices form a useful route from start to finish? A model can predict the next place well and still create revisits or a poor order. This thesis addresses that gap.

Transition: *This leads to a two-phase system and a two-benchmark evaluation.*

Skip if late: *In other words, local accuracy does not automatically produce global route quality.*

Slide 3 · 0:55 · One Engine, One Decoder, and Two Fair Evaluations

MEMORY: One engine and one decoder are evaluated on two strictly separate benchmarks.

The study has two phases. Phase 1 learns on Foursquare New York and ranks about 5,100 places, using HR at k, NDCG, and MRR. Phase 2 uses Frozen Rollout to build complete itineraries. Routes are compared internally on New York, then evaluated on the Flickr Benchmark under Chen 2016. Here the metrics are set-F1 and order-aware pairs-F1. The two tasks measure different outcomes, so their scores must never share one leaderboard.

Transition: *With that map in mind, let us open the prediction engine.*

Skip if late: *The rule is simple: compare only the same dataset, protocol, task, and metric.*

Slide 4 · 1:15 · The Engine Combines Places, Context, and User Preferences

MEMORY: Graph, context, sequence, and user identity produce the next-POI ranking.

The engine combines four components. A hybrid graph links places that are often visited consecutively or geographically close; a two-layer GCN turns it into POI features. The graph uses training-only co-visitation links and each place's ten nearest geographic neighbors. Distance and elapsed time add context. A GRU summarizes the visited sequence. A learned user embedding adds identity-based preference information; it is learned from visit history, not from a written persona. The GRU state and user embedding pass through an MLP that scores all 5,100 places. Every session contributes every prefix-to-next-place pair, giving roughly one hundred thousand training examples.

Transition: *Now we can ask how accurate this compact engine is.*

Skip if late: *The implemented context is limited to distance and elapsed time; weather and time of day remain future work.*

Slide 5 · 0:50 · Competitive, but Not State of the Art

MEMORY: HR at one is 0.187: an honest LSTM-STGCN-tier result.

On New York, HR at one is 0.187: the exact next place ranks first almost once in five among about 5,100 candidates. HR at ten is 0.588, so it appears in the top ten almost six times in ten. LSTM scores 0.130 and STGCN 0.180; our 0.187 is higher, while attention, transformer, and LLM methods remain ahead.

Transition: *The next question is how to turn these local scores into a route.*

Skip if late: *This is competitive in the LSTM-STGCN tier, but not state of the art.*

Slide 6 · 1:15 · Frozen Rollout Builds a Loop-Free Route

MEMORY: Frozen Rollout builds loop-free routes without retraining the engine.

Frozen Rollout receives the user, start, destination, and desired number of stops. At each step, the frozen engine scores every place. The decoder masks visited places, reserves the destination for the final position, selects the best remaining candidate, and repeats. This guarantees a loop-free route with the requested endpoints and length. Greedy decoding takes the best candidate; beam search keeps several partial routes. Both enforce the same constraints. No model weight changes and no additional training is needed: the same checkpoint now serves the itinerary task. The itinerary capability is added entirely at inference time.

Transition: *I then tested whether a model trained directly for itineraries could do better.*

Skip if late: *The headline method does not claim a global optimum or travel-time optimization.*

Slide 7 · 1:05 · Dense Supervision Beats a Purpose-Trained Model

MEMORY: Dense prefix supervision lets Frozen Rollout beat the Trained Pointer.

On 2,880 New York test routes of length at least three, Frozen Rollout reaches 0.289 pairs-F1. The Trained Pointer, trained end to end on whole routes, reaches 0.259, or 0.261 with context. Validation shows this is a genuine negative result, not a bug. The main explanation is supervision density: the engine sees about one hundred thousand prefix-target examples, while the Trained Pointer sees one whole trajectory per session. Beam search moves Frozen Rollout only to 0.290, so extra search cannot replace missing information in the scorer.

Transition: *However, these New York values cannot be compared directly with published itinerary scores.*

Skip if late: *Set-F1 of 0.609 suggests that finding places is easier than ordering them.*

Slide 8 · 0:55 · A Fair Comparison Requires the Same Benchmark

MEMORY: Reproducing Random within 0.021 validates the Flickr evaluation protocol.

Flickr city vocabularies are far smaller than New York, so the chance level is different. For a fair bridge, I reproduced Chen 2016: leave one trajectory out, give the first and last POI plus length, and evaluate loop-free routes of length at least three. Random is the key check because it has no modeling choices. Across five cities, my Random score differs from Chen by at most 0.021. This acts like a unit test for the complete evaluation protocol.

Transition: *Only after this check is a literature comparison meaningful.*

Skip if late: *That match indicates that the data, split, query, and pairs-F1 calculation are aligned.*

Slide 9 · 1:10 · The Benchmark Reveals Both Positioning and Cold-Start

MEMORY: Our methods reach the published scale, but cold-start limits identity-based personalization.

On the Flickr Benchmark, our methods span 0.23 to 0.59 pairs-F1 on the published scale. Markov is strongest among ours, reaching 0.587 in Glasgow. The Trained Pointer beats Random but trails Markov in every city, while SelfTrip and AR-Trip remain around 0.8. Our classical methods therefore reach the literature's scale, but the richer neural methods remain clearly ahead. Personalization is mixed: the user embedding changes pairs-F1 by plus 0.038 in Glasgow, minus 0.007 in Osaka, and minus 0.017 in Toronto. Under leave-one-out, many users offer too little history for a stable identity embedding, which is a cold-start effect.

Transition: *I will finish with the three results worth remembering.*

Skip if late: *A content- or persona-based signal is therefore a logical next step.*

Slide 10 · 0:40 · Three Results to Remember

MEMORY: Remember 0.187, 0.289 versus 0.259, and the validated Flickr bridge.

Three results matter. The engine reaches HR at one of 0.187 on 5,100 places. Frozen Rollout builds loop-free routes and beats the Trained Pointer, 0.289 versus 0.259. The Flickr Benchmark closely reproduces Chen's Random baseline, so the literature positioning is defensible. A trained local model can support itinerary generation when decoding constraints and evaluation boundaries are explicit. Thank you.

Transition: *That concludes the presentation and opens the discussion.*

Skip if late: *I am ready for your questions.*

Likely Jury Questions

Answer directly, give one piece of evidence, and stop. Return to the closing slide after the answer.

1. What is the main contribution of this thesis?

A reproducible context-aware next-POI engine, Frozen Rollout for converting its scores into loop-free routes without retraining, and a Flickr Benchmark that places the itinerary results on a literature-comparable scale. The contribution is the system, the integration experiment, and the rigorous evaluation - not a new state-of-the-art architecture.

2. Why treat next-POI prediction and itinerary generation as separate tasks?

Next-POI prediction ranks one future place, while itinerary generation must select and order an entire route. They therefore use different outputs and metrics: HR@k, NDCG, and MRR for next-POI; set-F1 and pairs-F1 for itineraries.

3. What does HR@1 = 0.187 actually mean, and why use a GRU?

It means the correct next venue is the top prediction in 18.7 percent of cases among about 5,100 candidates; it is not an itinerary-success rate. The GRU provides a lean, reproducible sequential backbone and produces an honest LSTM/STGCN-tier result.

4. How does Frozen Rollout generate an itinerary?

It repeatedly applies the frozen next-POI scorer to the current partial route. Already visited POIs are masked, the requested end POI is reserved for the final step, and greedy or beam selection continues until the requested route length is reached.

5. Why does the Trained Pointer lose to Frozen Rollout?

The engine learns from roughly 10^5 prefix-target pairs, whereas the Trained Pointer receives one whole-trajectory example per session. Frozen Rollout reuses a converged engine, while the Trained Pointer trains its encoder from scratch. Adding context improves the Trained Pointer only from 0.259 to 0.261.

6. How do you know the Trained Pointer result is not a training bug?

Validation pairs-F1 rose to a genuine peak before early stopping, and the context-enhanced variant was tested. The same data-size pattern appears on the Flickr Benchmark, where the Trained Pointer trails Markov. This supports a real negative result for the tested setup, not a universal claim about every itinerary model.

7. Why does beam search barely improve Frozen Rollout?

Beam search changes pairs-F1 only from 0.289 to 0.290. The much higher set-F1 of 0.609 shows that many correct places are found but their order remains imperfect. Searching harder over the same local scores cannot introduce information the scorer never learned.

8. Why can the NYC and Flickr results not be compared directly?

NYC contains about 5,100 POIs, while the Flickr cities contain only 27 to 88. Chance level and task difficulty differ dramatically. Valid comparison requires the same dataset, protocol, and metric.

9. How did you validate the Flickr evaluation harness?

I followed Chen 2016: leave-one-trajectory-out evaluation, first and last POIs plus route length given, routes of length at least three, loop-free output, and pairs-F1. Our Random results differ from Chen by at most 0.021 across five cities.

10. Does personalization actually help?

Conditionally. The user embedding improves Glasgow from 0.451 to 0.489, a +0.038 gain, but changes Osaka by -0.007 and Toronto by -0.017. Under leave-one-out, many users have too little remaining history to estimate an identity embedding reliably.

11. What are the main limitations?

The engine remains below neural and LLM state of the art; decoding cannot correct the scorer's myopia; context is limited to distance and elapsed-time gaps; decode-time elapsed time is assumed; and the next-POI experiment covers one city with offline evaluation only - no deployment or user study.

12. What is future work, and what was not implemented?

Future work includes weather and time-of-day context, persona or content-based cold-start representations, schedule-aware decoding with travel cost and opening hours, stronger pre-training, multi-city validation, and user studies. The orienteering or iterated-local-search solver was scoped but not implemented.

Never claim: state of the art; that HR@1 and pairs-F1 are comparable; that NYC and Flickr have the same difficulty; that personalization always helps; or that the orienteering / ILS method was implemented.